University of Colombo School of Computing

Faculty of Science

SCS2314 — Middleware Architecture

Assignment 4

SwiftTrack

Middleware Architecture for "SwiftLogistics"

Event-Driven Integration of Heterogeneous Legacy Systems

Group Members

Index Number	Name
23000821	R K K Jinendra
23000481	G C K S Gajanayake
23000694	J T D Jayakodi

Module: SCS2314 — Middleware Architecture
Lecturer: University of Colombo School of Computing
Submission Date: 28 February 2026
Technologies: Node.js, Express, RabbitMQ, MongoDB
Repository: https://github.com/Kalana2/SwiftLogistics

Abstract

This document presents the design and implementation of **SwiftTrack**, a middleware solution for Swift Logistics (Pvt) Ltd. that integrates three heterogeneous legacy systems: a SOAP/XML-based Client Management System (CMS), a RESTful Route Optimisation System (ROS), and a TCP/IP-based Warehouse Management System (WMS). The proposed architecture employs a microservices-based approach with RabbitMQ as the central message broker for asynchronous, event-driven communication. Key architectural contributions include an orchestration-based Saga pattern for distributed transaction management with compensating transactions, a lightweight service discovery registry with health-check monitoring, real-time WebSocket notifications via Socket.IO, and a comprehensive security layer with JWT authentication, Helmet.js, and rate limiting. The prototype demonstrates a complete order processing pipeline from client submission through CMS inventory verification, ROS route optimisation, and WMS driver assignment, with real-time status updates delivered to a pure HTML/CSS/JavaScript client portal. Two alternative architectures (ESB-centric and serverless) are evaluated and compared against the selected approach.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Problem Statement

Modern logistics organizations depend on multiple legacy systems—Content Management Systems (CMS) using SOAP/XML, Warehouse Management Systems (WMS) using TCP sockets, and Route Optimization Services (ROS) using REST APIs. These systems were designed independently and cannot communicate directly with each other or with modern web clients.

1.2 Solution Overview

SwiftTrack is an **event-driven middleware platform** that integrates these heterogeneous legacy systems through a unified microservice architecture. It uses RabbitMQ as a message broker to orchestrate asynchronous communication, MongoDB for persistence, and Socket.IO for real-time client notifications.

1.3 Key Objectives

1. Provide a unified REST API for all client interactions
2. Translate between disparate protocols (REST ↔ SOAP, REST ↔ TCP)
3. Ensure reliable message delivery with RabbitMQ
4. Support real-time order tracking via WebSocket
5. Implement the Saga pattern for distributed transaction management

2 System Architecture

2.1 Architectural Overview

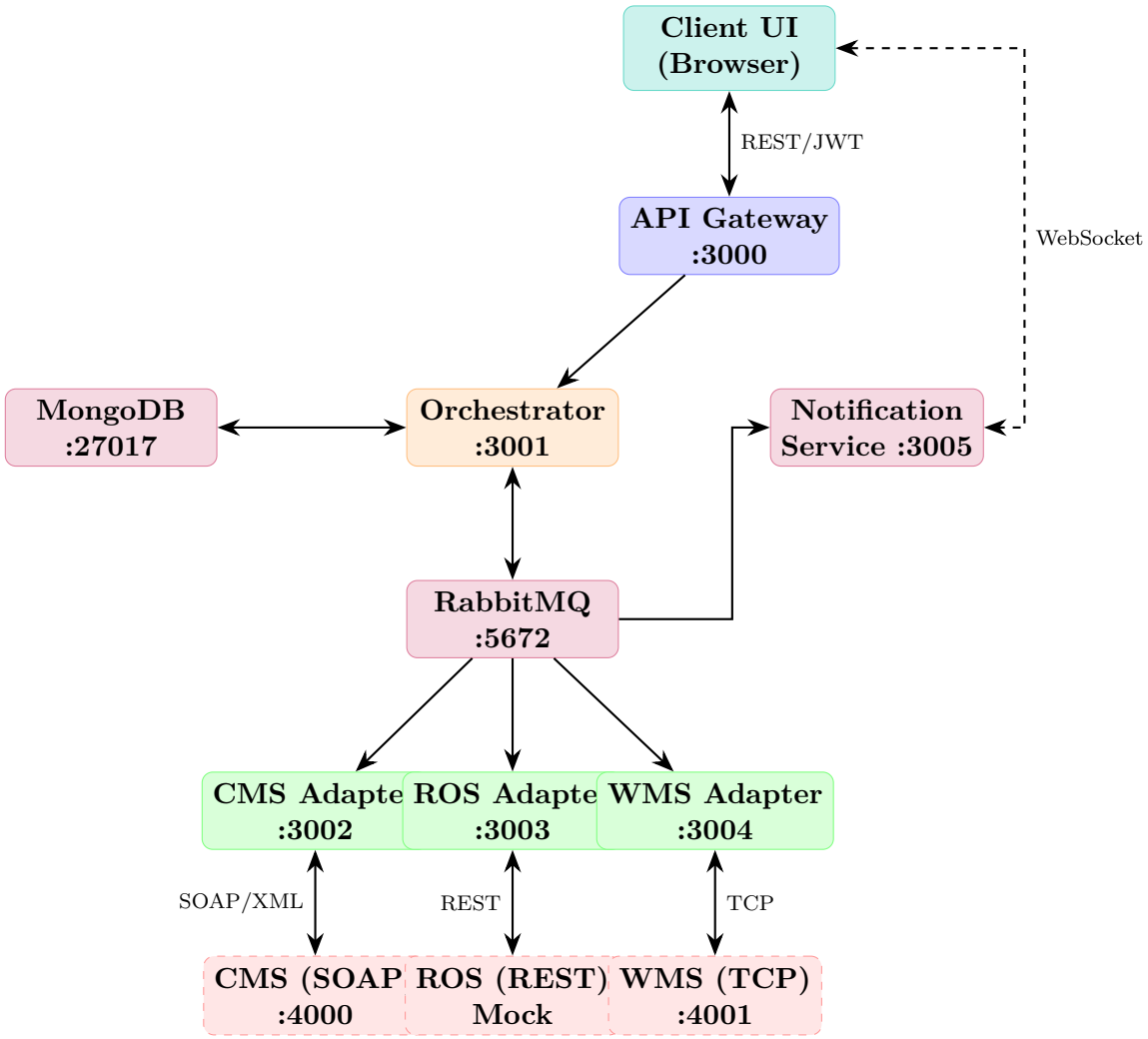

Figure 1: SwiftTrack System Architecture

2.2 Technology Stack

Table 1: Technology Stack

Component	Purpose	Technology
API Gateway	Authentication, routing, rate limiting	Express, JWT, Helmet
Orchestrator	Order lifecycle, business logic	Express, Mongoose
Message Broker	Async inter-service communication	RabbitMQ (AMQP)
Database	Order persistence	MongoDB
CMS Adapter	Legacy CMS integration	SOAP/XML (fast-xml-parser)
ROS Adapter	Route optimization	REST (Axios)
WMS Adapter	Warehouse management	TCP (net module)
Notification Svc	Real-time updates	Socket.IO
Client UI	Web dashboard	HTML, CSS, JavaScript

3 Service Descriptions

3.1 API Gateway (Port 3000)

The API Gateway is the single entry point for all client requests. It handles:

- **Authentication**: JWT token generation and verification
- **Authorization**: Role-based access control (client vs driver)
- **Rate Limiting**: 100 requests per minute per IP
- **CORS**: Cross-origin resource sharing for the client UI
- **Request Proxying**: Forwards validated requests to the Orchestrator

```
function authenticateJWT(req, res, next) {
  const authHeader = req.headers.authorization;
  const token = authHeader?.split(' ')[1];

  if (!token) return res.status(401).json({ error: 'No token' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    return res.status(403).json({ error: 'Invalid token' });
  }
}
```

Listing 1: JWT Authentication Middleware

3.1.1 API Endpoints

Table 2: API Gateway Route Map

Method	Path	Description
POST	/api/auth/login	Authenticate user, return JWT
POST	/api/auth/register	Register new user
GET	/api/orders	List customer orders
POST	/api/orders	Submit new order
GET	/api/orders/:id	Get order details
GET	/api/driver/manifest	Get driver delivery manifest
POST	/api/driver/delivery/:id/status	Update delivery status
GET	/health	Service health check

3.2 Orchestration Service (Port 3001)

The Orchestration Service is the **central coordinator** of the order lifecycle. It persists orders in MongoDB and publishes events to RabbitMQ for downstream adapters.

3.2.1 Order Lifecycle

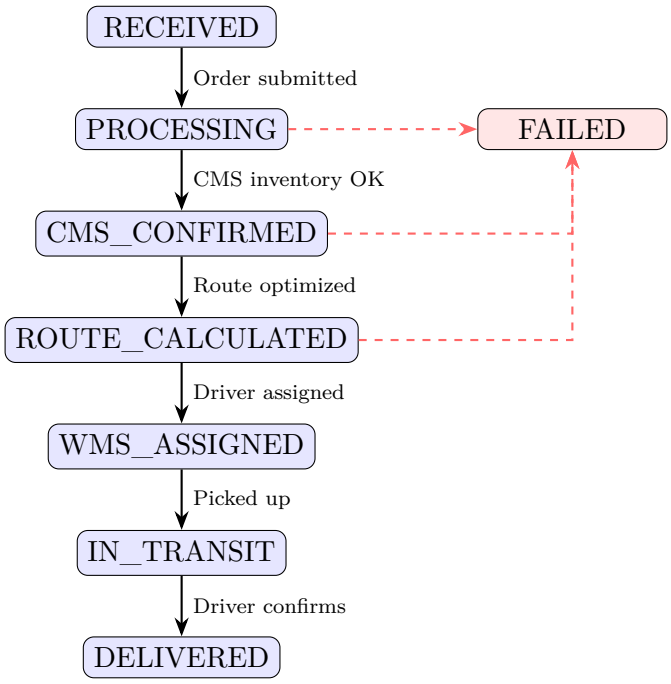

Figure 2: Order Status State Machine

3.2.2 MongoDB Schema

```
const orderSchema = new mongoose.Schema({
  orderId:    { type: String, required: true, unique: true },
  customerId: { type: String, required: true, index: true },
  items: [{
    sku:      { type: String, required: true },
    name:     String,
    quantity: { type: Number, min: 1 },
    price:    { type: Number, min: 0 }
  }],
  deliveryAddress: {
    street: String, city: String, state: String, zip: String
  },
  status: {
    type: String,
    enum: ['RECEIVED','PROCESSING','CMS_CONFIRMED',
           'ROUTE_CALCULATED','WMS_ASSIGNED',
           'IN_TRANSIT','DELIVERED','FAILED'],
    default: 'RECEIVED'
  },
  metadata: {
    cmsReference: String,
    routeId: String,
    warehouseId: String,
    driverId: String,
    estimatedDelivery: Date
  },
  statusHistory: [{ status: String, timestamp: Date,
                    message: String, source: String }]
}, { timestamps: true });
```

Listing 2: Mongoose Order Schema

3.3 RabbitMQ Message Architecture

Table 3: RabbitMQ Exchanges and Queues

Exchange	Type	Purpose	Routing Keys
order_exchange	topic	Route orders to adapters	order.cms, order.ros, order.wms
event_exchange	fanout	Broadcast events to all	* (all listeners)

Table 4: Message Queues

Queue	Purpose	Consumer
new_order_queue	New order submissions	Orchestrator
cms_request_queue	CMS inventory/confirm	CMS Adapter
ros_request_queue	Route calculation	ROS Adapter
wms_request_queue	Warehouse/driver assign	WMS Adapter
completed_order_queue	Completed orders	Orchestrator
notification_queue	Real-time notifications	Notification Service

3.3.1 Event Types

```
const EventTypes = {
  ORDER_RECEIVED:     'order.received',
  ORDER_PROCESSING:   'order.processing',
  ORDER_COMPLETED:    'order.completed',
  ORDER_FAILED:       'order.failed',
  CMS_SUCCESS:        'cms.success',
  CMS_FAILURE:        'cms.failure',
  ROS_SUCCESS:        'ros.success',
  ROS_FAILURE:        'ros.failure',
  WMS_SUCCESS:        'wms.success',
  WMS_FAILURE:        'wms.failure',
  NOTIFICATION_SEND:  'notification.send'
};
```

Listing 3: Event Type Definitions

4 Adapter Layer

4.1 CMS Adapter (Port 3002) — SOAP/XML Integration

The CMS Adapter communicates with the legacy Content Management System over **SOAP (Simple Object Access Protocol)**. It translates JSON messages from RabbitMQ into XML SOAP envelopes and parses XML responses back.

4.1.1 SOAP Request Flow

1. Receives `order.cms` message from RabbitMQ
2. Builds XML SOAP envelope for inventory check
3. Sends HTTP POST to CMS at `/cms/soap`
4. Parses XML response (`fast-xml-parser`)
5. If inventory available: sends confirmation SOAP request
6. Updates order status to `CMS_CONFIRMED`
7. Publishes `CMS_SUCCESS` event

```xml
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:cms="http://swifttrack.com/cms/2026">
  <soap:Body>
    <cms:CheckInventoryRequest>
      <cms:OrderId>ORD-2026-0001</cms:OrderId>
      <cms:Items>
        <cms:Item>
          <cms:SKU>PROD001</cms:SKU>
          <cms:Quantity>2</cms:Quantity>
        </cms:Item>
      </cms:Items>
    </cms:CheckInventoryRequest>
  </soap:Body>
</soap:Envelope>
```

Listing 4: SOAP Inventory Check Request

4.2 ROS Adapter (Port 3003) — REST Integration

The Route Optimization Service adapter handles route calculation. It supports both external API calls (e.g., Mapbox, Google Maps) and a built-in mock calculation for development.

4.2.1 Mock Route Calculation

```javascript
function calculateMockRoute(origin, destination) {
  const distance = Math.random() * 50 + 5;   // 5-55 km
  const duration = distance * 2.5;            // ~2.5 min/km
  return {
    routeId: `ROUTE_MOCK_${Date.now()}`,
    distance: Math.round(distance * 10) / 10,
    duration: Math.round(duration),
    estimatedDelivery: new Date(
      Date.now() + duration * 60 * 1000
    ).toISOString()
  };
}
```

Listing 5: Mock Route Calculation

4.3 WMS Adapter (Port 3004) — TCP Socket Integration

The WMS Adapter communicates with the Warehouse Management System via a **raw TCP socket connection**. Messages are newline-delimited JSON.

4.3.1 TCP Protocol

Table 5: WMS TCP Message Types

Message Type	Purpose
ASSIGN_WAREHOUSE	Assign nearest warehouse based on delivery destination
ASSIGN_DRIVER	Assign available driver from warehouse
UPDATE_STATUS	Update delivery status (DELIVERED, FAILED)

```
class WMSClient {
  constructor(host, port) {
    this.client = new net.Socket();
    this.buffer = '';
    this.pending = new Map();
  }

  async sendMessage(message) {
    return new Promise((resolve, reject) => {
      const id = Date.now().toString();
      this.pending.set(id, { resolve, reject });
      this.client.write(JSON.stringify(message) + '\n');
    });
  }
}
```

Listing 6: TCP Client Connection

5 Notification Service (Port 3005)

The Notification Service provides **real-time WebSocket communication** to connected clients using Socket.IO. It consumes events from the RabbitMQ `event_exchange` and broadcasts updates.

5.1 Notification Channels

- **Customer notifications**: Order status updates for a specific customer
- **Order-specific notifications**: Updates for subscribed orders
- **Driver notifications**: New assignment and route updates

```
socket.on('register', ({ userId, customerId, role }) => {
  if (customerId) socket.join(`customer:${customerId}`);
  if (role === 'driver') socket.join(`driver:${userId}`);
});
```

```
5
6  // Broadcast order update
7  io.to(`customer:${customerId}`).emit('order:update', {
8    orderId, status, message, timestamp
9  });
```

Listing 7: Socket.IO Event Broadcasting

6 Client UI

The frontend is a **pure HTML/CSS/JavaScript** single-page application with no build tools or frameworks. It uses:

- Hash-based routing (`#/login`, `#/dashboard`)
- JWT-authenticated API calls
- Self-contained inline SVG icons (no CDN dependency)
- Automatic mock API fallback when backend is offline

6.1 Pages

Table 6: UI Pages

Page	Features
Login	Email/password form, JWT handling, role-based redirect
Client Dashboard	Order statistics, order table, new order modal, order detail panel
Driver Dashboard	Delivery manifest, deliver/fail actions, real-time updates

7 Data Flow

7.1 Complete Order Processing Flow

1. **Client** sends POST `/api/orders` with JWT token
2. **API Gateway** validates JWT, forwards to Orchestrator
3. **Orchestrator** persists order in MongoDB (status: `RECEIVED`)
4. **Orchestrator** publishes to `order_exchange` with key `order.cms`
5. **CMS Adapter** consumes message, sends SOAP request to CMS
6. **CMS** checks inventory, returns SOAP response
7. **CMS Adapter** updates Orchestrator via REST (status: `CMS_CONFIRMED`)
8. **Orchestrator** publishes to `order_exchange` with key `order.ros`

9. **ROS Adapter** calculates route, updates status: `ROUTE_CALCULATED`
10. **Orchestrator** publishes to `order_exchange` with key `order.wms`
11. **WMS Adapter** sends TCP message to WMS for warehouse + driver assignment
12. **WMS** assigns, returns via TCP; adapter updates status: `WMS_ASSIGNED`
13. **Notification Service** receives events, pushes via WebSocket to client
14. **Driver** picks up package → status: `IN_TRANSIT`
15. **Driver** confirms delivery → status: `DELIVERED`

8 Alternative Architecture Proposals

Before selecting the final microservices + RabbitMQ architecture, two alternative approaches were evaluated.

8.1 Alternative 1: Enterprise Service Bus (ESB)

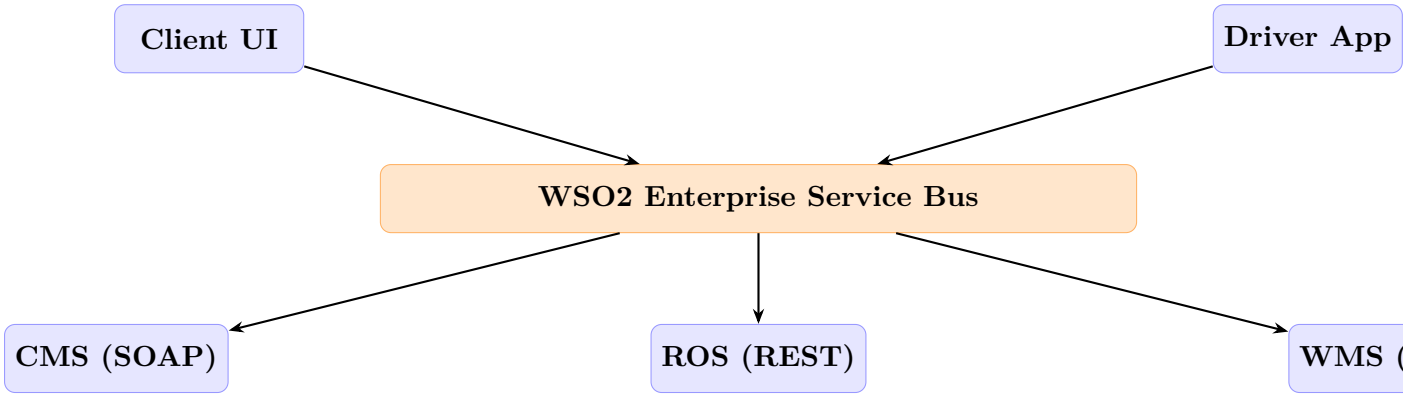

Figure 3: Alternative 1: ESB-Centric Architecture

Approach: Use a centralized Enterprise Service Bus (e.g., WSO2 ESB or MuleSoft Anypoint) as the integration backbone. All protocol translation, message routing, and orchestration would be handled by the ESB.

Pros: Built-in protocol mediation (SOAP, TCP, REST), visual flow designer, enterprise-grade monitoring, built-in transaction management.

Rejected because:

- **Single point of failure**: All communication flows through one centralized bus
- **Operational complexity**: Requires significant Java/XML expertise to configure and maintain
- **Scalability limitations**: Vertical scaling of the ESB becomes a bottleneck under high load (e.g., Black Friday events)
- **Vendor lock-in**: Deep coupling to the ESB vendor's APIs and configuration

8.2 Alternative 2: Serverless (FaaS) Architecture

Approach: Deploy each integration point as a serverless function (e.g., AWS Lambda) with an event bus (e.g., AWS SQS/EventBridge) for messaging.

Pros: Zero infrastructure management, automatic scaling, pay-per-invocation cost model, managed message queues.

Rejected because:

- **Vendor lock-in**: Tight coupling to AWS/GCP/Azure services, violating the open-source requirement
- **Cold start latency**: Real-time notification and TCP persistent connections are poorly suited for serverless
- **TCP limitations**: WMS requires persistent TCP socket connections, which are incompatible with stateless function execution
- **Cost unpredictability**: High-volume order processing (Black Friday) could lead to unexpected costs

8.3 Rationale for Selected Architecture

The **microservices + RabbitMQ** architecture was selected because it:

1. Is fully open-source with no vendor lock-in
2. Allows independent scaling of each adapter service
3. Supports persistent TCP connections (WMS) via dedicated Node.js processes
4. Provides reliable asynchronous messaging with RabbitMQ durability
5. Enables real-time notifications via Socket.IO alongside REST APIs
6. Follows the assignment's emphasis on middleware architectural patterns

9 Transaction Management — Saga Pattern

9.1 The Distributed Transaction Problem

A single order triggers operations across three independent systems: CMS (inventory), ROS (routing), and WMS (warehouse/driver). Traditional ACID transactions cannot span these heterogeneous systems. Instead, we implement the **Orchestration-based Saga pattern**.

9.2 Saga Execution Flow

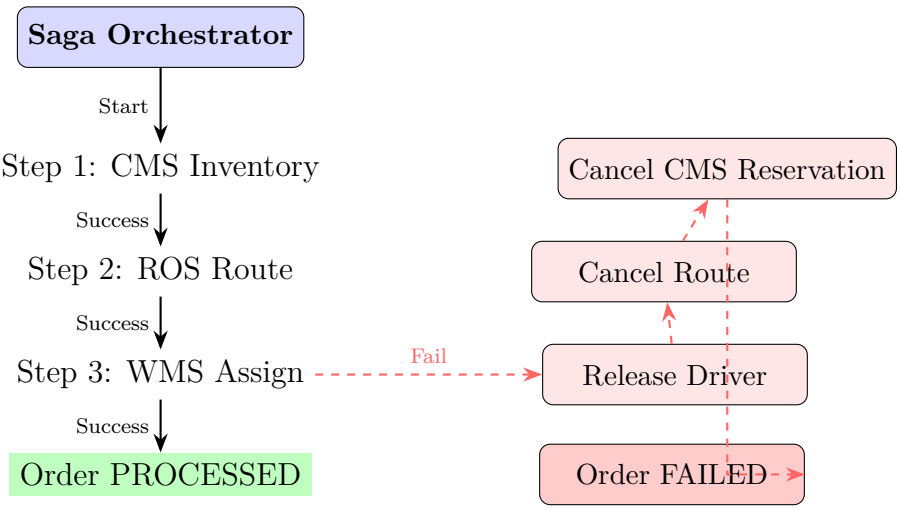

Figure 4: Saga Pattern — Sequential Execution with Compensating Transactions

9.3 Compensation Actions

Table 7: Saga Step Compensation Matrix

Step	Action	Compensation on Failure
CMS Inventory	Check & reserve inventory via SOAP	Release reserved inventory (CMS_CANCELLED)
ROS Route	Calculate optimized route via REST	Cancel route booking (ROUTE_CANCELLED)
WMS Assignment	Assign warehouse & driver via TCP	Release driver & warehouse slot (WMS_RELEASED)

```
class SagaOrchestrator {
  addStep(name, executeFn, compensateFn) { ... }

  async execute(context) {
    for (const step of this.steps) {
      try {
        await Promise.race([
          step.execute(context),
          timeout(30000) // 30s timeout per step
        ]);
        completedSteps.push(step);
      } catch (error) {
        // Compensate all completed steps in reverse
        for (const s of completedSteps.reverse()) {
          await s.compensate(context);
        }
        return { success: false, failedStep: step.name };
      }
    }
```

```
    return { success: true, sagaLog };
  }
}
```

Listing 8: Saga Orchestrator Implementation

9.4 Recovery Mechanisms

1. **Timeout-based failure**: Each saga step has a 30-second timeout. If an adapter doesn't respond, the step is treated as failed and compensation begins.
2. **Idempotent operations**: Adapter operations are designed to be idempotent, allowing safe retries.
3. **Saga log persistence**: The full saga execution log is stored in MongoDB alongside the order, enabling audit and manual recovery.
4. **Dead letter queues**: Failed RabbitMQ messages are routed to dead letter queues for manual inspection.

10 Service Discovery

10.1 Service Registry Architecture

SwiftTrack implements a lightweight **self-registration service registry** to manage service locations in the distributed environment.

```
class ServiceRegistry {
  register(name, host, port) { ... }
  getService(name)  // returns healthy instance URL
  startHealthChecks(interval = 10000)  // polls /health
}
```

Listing 9: Service Registry

10.2 Health Check Mechanism

- Each service exposes a `GET /health` endpoint returning status, uptime, and connection states
- The registry polls all endpoints every 10 seconds
- After 3 consecutive failures, a service is marked `DOWN`
- The API Gateway exposes `GET /api/services` for runtime discovery

Table 8: Service Registry Health States

State	Condition	Behaviour
UP	Health check passes	Included in service resolution
DOWN	3+ consecutive failures	Excluded from resolution, logged as warning

11 Proof of Delivery

Drivers can capture proof of delivery when marking a package as delivered. The system supports:

- **Digital Signature**: Canvas-based signature capture (stored as base64)
- **Photo Evidence**: Camera capture of delivered package
- Data stored in MongoDB within the order's `proofOfDelivery` field

```
proofOfDelivery: {
  photo:      String,    // base64 encoded image
  signature:  String,    // base64 canvas data
  capturedAt: Date,
  capturedBy: String     // driver ID
}

// API endpoint
POST /api/driver/delivery/:orderId/proof
Body: { photo: "base64...", signature: "base64..." }
```

Listing 10: Proof of Delivery Schema

12 Design Patterns

12.1 Adapter Pattern

Each legacy system has a dedicated adapter that translates between the internal JSON message format and the external protocol (SOAP, TCP, REST). This isolates protocol-specific logic from the business logic.

12.2 Saga Pattern (Orchestration-based)

The order processing uses an orchestration-based saga with compensating transactions. The `SagaOrchestrator` class manages sequential step execution and automatic rollback on failure (see Section 8).

12.3 Event-Driven Architecture

Services communicate asynchronously through RabbitMQ using topic and fanout exchanges. This provides loose coupling, fault tolerance, and independent scalability.

12.4 API Gateway Pattern

A single entry point handles cross-cutting concerns (authentication, rate limiting, CORS, logging) and proxies requests to internal services.

12.5 Service Registry Pattern

Services self-register with a lightweight registry on startup. The registry monitors health and provides dynamic URL resolution (see Section 9).

13 Security

13.1 Authentication & Authorization

- **JWT Authentication**: Stateless token-based auth with configurable expiry (default 24h). Tokens contain user ID, role, and customer/driver ID.
- **Role-Based Access Control (RBAC)**: Client and driver roles with route-level enforcement. Drivers cannot access client endpoints and vice versa.

13.2 Transport Security

- **Helmet.js**: Adds 11 HTTP security headers including X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security (HSTS), and Content-Security-Policy
- **CORS**: Strictly restricted to configured client origin only
- **TLS/HTTPS**: Production deployment uses TLS certificates for all client-facing endpoints

13.3 Rate Limiting & DDoS Protection

- **Rate Limiting**: 100 requests/minute per IP via `express-rate-limit`, crucial for handling peak events (Black Friday, Avurudu sales)
- **Sliding window**: Rate limits use a sliding window algorithm to prevent burst attacks

13.4 Input Validation & Sanitization

- **Server-side validation**: All inputs validated via `express-validator` before processing
- **MongoDB injection prevention**: Mongoose ODM with schema validation prevents NoSQL injection
- **XML injection prevention**: SOAP payloads are constructed programmatically (not via string concatenation) using `fast-xml-parser`

13.5 Secret Management

- All secrets (JWT keys, database URIs, API keys) stored in `.env` files
- `.env` files are excluded from version control via `.gitignore`
- Production uses environment variables injected via Docker or orchestration tools

14 Deployment

14.1 Docker Compose

All services are containerized via Docker Compose with a shared bridge network:

```
services:
  mongodb           # MongoDB 6     data persistence
  rabbitmq          # RabbitMQ 3     message broker
  api-gateway       # Express API Gateway
  orchestration-service  # Order orchestrator
  cms-adapter       # SOAP/XML adapter
  ros-adapter       # Route optimization adapter
  wms-adapter       # TCP warehouse adapter
  notification-service   # Socket.IO notifications
  cms-mock          # Mock CMS SOAP server
  wms-mock          # Mock WMS TCP server
```

Listing 11: Docker Compose Service List

14.2 Port Allocation

Table 9: Service Port Map

Service	Port	Protocol
API Gateway	3000	HTTP/REST
Orchestrator	3001	HTTP/REST
CMS Adapter	3002	HTTP (health)
ROS Adapter	3003	HTTP (health)
WMS Adapter	3004	HTTP (health)
Notification Service	3005	HTTP + WebSocket
CMS Mock	4000	HTTP/SOAP
WMS Mock	4001	TCP
MongoDB	27017	MongoDB Wire
RabbitMQ	5672 / 15672	AMQP / Management UI
Client UI	5173	HTTP (static)

15 Conclusion

SwiftTrack demonstrates a complete event-driven middleware solution for integrating heterogeneous legacy systems in a logistics domain. The architecture provides:

- **Protocol translation** between SOAP, TCP, and REST via dedicated adapters
- **Reliable messaging** via RabbitMQ with topic and fanout exchanges
- **Real-time tracking** through Socket.IO WebSocket notifications
- **Distributed transaction management** via the Saga pattern with compensating transactions

- **Service discovery** through a health-check-based registry
- **Proof of delivery** with digital signature and photo capture
- **Scalability** through independent microservice deployment
- **Security** with JWT, Helmet, RBAC, and rate limiting

The system successfully bridges the gap between legacy enterprise systems and modern web clients, enabling real-time logistics tracking without modifying the underlying legacy infrastructure.